

Beaver's Choice Paper Company — Multi-Agent System

Reflection Report

Framework	smolagents 1.26
Model	gpt-4o-mini at temperature=0.1 , via the Vocareum OpenAI-compatible proxy (https://openai.vocareum.com/v1)
Agents	4 — one orchestrator + three workers, against a permitted maximum of five
Diagram	diagram/beavers_choice_workflow.png
Evaluation output	test_results.csv , full agent trace in run_full.log

1. The agent workflow and why it looks like this

1.1 The four agents

The system is a strict hierarchy: one orchestrator that owns no tools and never touches the database, and three worker agents that each own one business concern.

#	Agent	Type	Owns	Explicitly not its job
1	orchestrator_agent	CodeAgent	Parsing the request into line items, delegation order, and writing the single customer reply	Never touches the database
2	inventory_agent	ToolCallingAgent	Stock truth, supplier lead times, reorder decisions	Never prices, never sells
3	quoting_agent	ToolCallingAgent	Pricing and the bulk-discount ladder	Never checks stock, never sells
4	sales_agent	ToolCallingAgent	Committing or refusing transactions	Never sets the price

Why these particular boundaries. I did not split the work by "task the customer asks about" but by *failure mode*. A wrong stock reading oversells goods we do not have; a wrong price quietly destroys margin; a wrong database write corrupts the ledger. Those are three different kinds of damage with three different remedies, so each gets one owner. When a response is wrong, the trace in `run_full.log` points at exactly one agent.

Why `CodeAgent` for the orchestrator and `ToolCallingAgent` for the workers. Each worker does one narrow job and mostly needs to call a tool and report the result, so structured JSON tool calls are

cheaper, faster and leave less room to improvise. The orchestrator has to loop over several line items per request and branch on each one's availability, so it benefits from `CodeAgent`, which reasons in Python and can hold intermediate values across steps.

Why four agents and not five. The rubric permits five and I deliberately used four. The obvious fifth candidate — a "customer communications" agent that turns the workers' findings into prose — would duplicate the orchestrator's final step, add a round-trip of latency and token cost, and give two agents overlapping responsibility. Fewer agents with cleaner boundaries beat more agents with blurred ones.

1.2 The decision flow

For each request the orchestrator runs a fixed five-step procedure:

1. **Understand.** Extract each product line with its quantity, and the date the customer needs delivery by. If no date is stated, assume the request date plus `DEFAULT_LEAD_DAYS` (10).
2. **Availability.** Delegate all line items to `inventory_agent` in one call. It returns a per-line verdict of `AVAILABLE`, `RESTOCKED`, or `UNAVAILABLE` with a reason.
3. **Price.** Delegate the resolved item names and quantities to `quoting_agent`, which returns a final total and the discount rate applied.
4. **Fulfil.** Send only the `AVAILABLE` and `RESTOCKED` lines to `sales_agent`. Lines marked `UNAVAILABLE` are never sent, so they cannot be sold by accident. If no line is fulfillable, `sales_agent` is not called at all.
5. **Reply.** Compose one customer-facing message.

The important property is that **the decision to decline is structural, not prompted**. A line is declined because the orchestrator's Python control flow never routes it to `sales_agent`, not because a prompt asked the model to be careful.

1.3 The design principle: the LLM decides *which*, Python decides *what*

Every number in the system — the discount tier, the line total, the cash-reserve check, the delivery-deadline check, the stock comparison — is computed in Python inside a tool. The model only chooses which tool to call and with what arguments. It never performs arithmetic and never invents a date.

Business policy therefore lives in named constants, not prose:

```
SUPPLIER_COST_RATIO    = 0.50      # cost of goods, as a fraction of retail
MIN_CASH_RESERVE        = 5_000.00 # a restock may never breach this
BULK_DISCOUNT_TIERS   = [(10_000, 0.15), (5_000, 0.12),
                           (1_000, 0.10), (500, 0.05), (100, 0.02)]
RESTOCK_BUFFER_MULTIPLIER = 1.5     # order the shortfall plus a safety buffer
DEFAULT_LEAD_DAYS       = 10       # assumed deadline when none is stated
```

Two of these are assumptions I imposed on the starter data and should be stated plainly. The starter ships only a retail `unit_price` per item, so `SUPPLIER_COST_RATIO = 0.50` models a 50% cost of goods; changing that single constant models a different margin. `MIN_CASH_RESERVE` is a solvency floor of my own choosing — the company must not spend itself to zero to win one order.

1.4 The bug that shaped the architecture

The first working version gave `inventory_agent` separate `check_stock_level` and `place_restock_order` tools and told it, in its instructions, to restock when stock was short. In testing it did something worse than fail: it checked stock, observed a shortfall, called `check_supplier_delivery` — and then **never placed the order**, while the orchestrator went on to quote the customer a price and promise a delivery date for goods that had never been bought.

The fix was not a sterner prompt. It was collapsing `check` → `decide` → `order` into a single atomic tool, `ensure_stock_available`, which resolves the item name, measures the shortfall, sizes the restock, applies both guards, and returns one of three verdicts that the orchestrator branches on:

```
AVAILABLE    - ships from stock
RESTOCKED    - was N units short; M units ordered, arriving <date>, in time
UNAVAILABLE  - short and cannot be restocked, because <reason>
```

The generalisable lesson: **when a multi-step decision must always happen, encode it in one tool rather than hoping the model chains the steps**. This is the single most consequential design decision in the project, and it is why the inventory agent's toolbox has one starred primary tool alongside six diagnostic ones.

1.5 Handling the starter data as it actually is

Two properties of the provided code shaped the tools:

- `search_quote_history` joins its search terms with `AND`. Passing four keywords almost always returns nothing. `find_similar_quotes` therefore retries with progressively fewer terms — all terms, then each term alone — rather than reporting "no precedent found" and letting the model guess.
- The seeded `inventory` table covers only 18 of the 44 catalog items. Pricing and the catalog listing therefore read the starter's own `paper_supplies` list rather than the seeded table, because an item we do not currently hold is still an item we can quote and reorder. Stock levels still come from `get_stock_level` against the transactions ledger.

Item-name resolution is also done in Python, not by the model (`_resolve_item_name`), and it is deliberately ordered from certain to speculative — refusing rather than guessing:

0. reject outright if any word is a known non-catalog product term (`UNSUPPORTED_PRODUCT_TERMS`);
1. exact, then case-insensitive exact;
2. an explicit synonym from `CATALOG_ALIASES`;
3. the longest catalog name contained in the phrasing, so "*heavy cardstock (white)*" finds `Cardstock` and "*A4 glossy paper*" finds `Glossy paper` rather than `A4 paper`;
4. the longest synonym contained in the phrasing, so a trailing qualifier such as "*table napkins (white)*" still resolves;
5. a fuzzy match, but only above `RESOLVER_FUZZY_CUTOFF` (0.82) and never across a paper-size boundary.

Steps 0 and 5 exist for the same reason, and section 4.8 explains what happened without them: a wrong resolution here does not fail loudly — it sells the customer a product they never asked for and bills them for it.

2. Tools and helper-function coverage

All seven required helper functions from the starter code are used in at least one tool definition.

Starter helper	Used by tool(s)	Owning agent(s)
<code>create_transaction</code>	<code>ensure_stock_available</code> , <code>place_restock_order</code> , <code>record_sale</code>	inventory, sales
<code>get_all_inventory</code>	<code>inventory_snapshot</code>	inventory
<code>get_stock_level</code>	<code>ensure_stock_available</code> , <code>check_stock_level</code> , <code>calculate_quote</code> , <code>record_sale</code>	inventory, quoting, sales
<code>get_supplier_delivery_date</code>	<code>ensure_stock_available</code> , <code>place_restock_order</code> , <code>check_supplier_delivery</code>	inventory, sales
<code>get_cash_balance</code>	<code>ensure_stock_available</code> , <code>place_restock_order</code> , <code>check_cash_balance</code>	inventory
<code>generate_financial_report</code>	<code>company_financial_report</code>	sales
<code>search_quote_history</code>	<code>find_similar_quotes</code>	quoting

`get_all_inventory` , `get_cash_balance` and `generate_financial_report` are assigned as internal reporting and health-check tools: `company_financial_report` is run by `sales_agent` before orders above \$5,000, and `check_cash_balance` funds the restock decision. Their figures are marked internal in the tool docstrings and are never shown to a customer.

3. Customer-facing transparency and safety

Four mechanisms keep the customer output complete but not leaky:

1. **Completeness.** The orchestrator's reply specification requires each item and quantity, the total price, the discount rate *and the order size that earned it*, the expected availability date, and — for any declined line — a specific reason.
2. **Justification.** Because `calculate_quote` returns the discount tier alongside the total, the reply can always explain *why* a price is what it is, rather than asserting a number.
3. **Confidentiality.** `QUOTING_AGENT_RULES` forbids mentioning supplier cost, margin, or cash position; `company_financial_report` 's docstring marks its figures internal; the orchestrator is told never to reveal balances, tool output, error messages, or the names of its internal agents.
4. **No leaked failures.** `handle_customer_request` catches every exception, logs the real cause to stdout for us, and returns a neutral message to the customer. A Python traceback can never reach `test_results.csv` .

5. **Money is never authored by the model.** `_enforce_authoritative_pricing` reads back the sales this request actually wrote to the ledger, replaces any money figure in the prose that does not match one of them, and appends an itemised *Confirmed order* block generated from the ledger. See section 4.7.
6. **A deterministic backstop on the vocabulary.** `_sanitize_customer_reply` runs on the final message before it leaves the system, rewriting the internal status codes (`AVAILABLE` , `RESTOCKED` , `UNAVAILABLE` , `SALE COMPLETED` , `SALE REJECTED`) into plain English and logging any remaining internal term for audit. Rules 1–3 are instructions and therefore advisory; this one is code. See section 4.6.

4. Evaluation results

The system was run against all 20 requests in `quote_requests_sample.csv` . Results are in `test_results.csv` ; the full agent trace is in `run_full.log` . Four scripts reproduce the checks below: `analyze_results.py` (rubric thresholds), `audit_prices.py` (every sale matches its quote), `audit_replies.py` (no internal vocabulary reaches a customer, and every price shown is backed by a recorded sale) and `audit_reconciliation.py` (the customer story and the ledger agree).

4.1 Run summary

Metric	Value	Rubric threshold
Requests processed	20 / 20	all
Requests that changed the cash balance	18	≥ 3
Requests with at least one recorded sale	18	≥ 3
Requests left entirely unfulfilled	2 (#15, #19)	≥ 1
Requests containing at least one declined line	9	—
Sales priced differently from their quote	0 / 43	—
Prices shown to a customer not backed by the ledger	0 / 20	—
Cash deltas that fail to reconcile with the ledger	0 / 20	—
Sales of an item the customer did not ask for	0	—
Internal status codes leaked to a customer	0 / 20	—
Runtime errors / fallback responses	0	—

Financial position	Start	End	Change
Cash balance	\$45,059.70	\$46,759.74	+\$1,700.04
Inventory value	\$4,940.30	\$4,338.70	−\$601.60
Total assets	\$50,000.00	\$51,098.44	+\$1,098.44

Beneath those figures the agents wrote 71 transactions: **43 sales lines** totalling **\$3,916.36** across **16 distinct catalog items**, and **28 supplier restock orders** totalling **\$2,216.32** — a **43.4% net margin** on

the period's trading. A further **12 restock orders were refused by a guard** before any money moved.

4.2 Fulfilled orders

Request 20 (2025-04-17) is the clearest demonstration of the system working end to end, and is discussed further in section 4.8. The customer asked for 5,000 flyers, 2,000 posters and 10,000 tickets. Flyers and posters sold at their full requested quantities for \$660.00 and \$450.00; tickets were declined because Beaver's Choice does not sell them. The reply's total, \$1,110.00, equals the ledger exactly:

```
Confirmed order
- 2000 x Poster paper - $450.00 (10% bulk discount)
- 5000 x Flyers - $660.00 (12% bulk discount)
Order total: $1,110.00
```

Request 3 (2025-04-04) shows the same discipline on a subtler case: 10,000 sheets of A4 paper sold at a 15% discount and 500 reams of printer paper at 5%, while "A3 paper" was declined — "*not part of our inventory*" — because the company does not stock that size and the system will not quietly substitute A4.

Request 14 (2025-04-09) is a partial fulfilment: two lines sold, one declined with a stated reason, and a ledger-generated confirmation closing the message.

A request's cash delta is *net*: sales minus any restock bought to serve it.

4.3 Declined orders

Two requests were declined outright, and each reason traces to a specific code guard rather than a model judgement:

- **Request 15** — a large shortfall on A4 white paper and on A3 colored paper, neither restockable before the 15 April deadline, plus "*cardboard for signage*", which the catalog does not carry. The first two are the `_restock` delivery-deadline guard; the third is `_resolve_item_name` refusing to force a match onto a real product.
- **Request 19** — three lines critically short, none restockable before the 20 April deadline.

A further nine requests declined at least one line while fulfilling others.

4.4 Strengths

1. **The customer story and the ledger agree.** Every cash delta reconciles with its slice of the transaction ledger, and every *Confirmed order* total equals both its own line items and the sales recorded (`audit_reconciliation.py`).
2. **Quote and ledger cannot diverge.** All 43 recorded sales match the price the customer was quoted (`audit_prices.py`).
3. **Every price a customer sees is backed by a recorded sale** (`audit_replies.py`), enforced by code rather than by instruction.
4. **No item is ever substituted for the one requested.** Terms the catalog does not carry are refused before any similarity scoring.

5. **No internal vocabulary reaches customers**, verified across all 20 replies.
6. **The system refuses rather than overselling**. No transaction was written for stock that did not exist.
7. **Refusals are specific and actionable** — units short, supplier delivery against the deadline, or an item not carried.
8. **Trading was profitable and solvent**. Cash never approached the \$5,000 reserve; the period closed \$1,700.04 up at a 43.4% margin.
9. **Zero failures**. No request hit the exception fallback.

4.5 First defect: the ledger trusted the model

An early full run passed every rubric threshold and closed \$1,899.57 up — and was wrong. Auditing each recorded sale against the price the customer had been quoted revealed **3 of 40 sale lines billed at a price the customer was never quoted**: two overcharged by \$190 each (a \$47.50 line written to the ledger at \$237.50) and one undercharged — a net overcharge of \$356.48.

`calculate_quote` computed the price deterministically, but `record_sale` accepted `total_price` as an argument and wrote whatever it was given, so the agreed figure had to survive a hop from `quoting_agent` through the orchestrator to `sales_agent`.

`record_sale` now **re-derives** the line total from the catalog price and the discount ladder, treats the caller's figure as a claim, and logs any disagreement as a `[price correction]`. In a later run that guard fired three times in a single request, once against a claimed \$1,125.00 on a line worth \$5.50 — so the corruption is real and recurrent, not a one-off.

4.6 Second defect: internal vocabulary leaked to a customer

The next run priced the ledger correctly but leaked differently: request 17 rendered its declined lines as "*A4 white printer paper: UNAVAILABLE*", passing an internal verdict token to the customer three times in one message.

The fix has two layers. The orchestrator's rules now name the five status codes as internal and give the phrasing to use instead; and `_sanitize_customer_reply` rewrites any surviving all-capitals token before the message leaves `handle_customer_request`, logging any other internal term for audit.

4.7 Third defect: the reply was priced by the model, not the ledger

Fixing the ledger did not fix the customer. In a later run, request 17's reply told the customer "\$237.50 ... \$237.50 ... total \$475.00" while the ledger, correctly, charged **\$95.00**. `calculate_quote` had returned \$47.50 per line; the model corrupted it on the hop to `sales_agent`; `record_sale`'s guard caught it and charged correctly — and the orchestrator then wrote the *corrupted* figure into the reply. The guard had protected the ledger and left the customer-facing message exposed, and `audit_prices.py` reported clean because it only ever checked the ledger.

`handle_customer_request` now marks the ledger position before the run, reads back exactly the sales that run produced, replaces any money figure in the prose that does not match one of them with an em dash, and appends an itemised *Confirmed order* block generated from the ledger. In this run that guard

intervened on two replies (#6 and #12), each time removing a model-computed total that disagreed with the ledger.

4.8 Fourth defect: fuzzy matching sold products nobody ordered

The most serious defect was found in review, and it is the reason for the resolver described in section 1.5.

`_resolve_item_name` ended in a `difflib` fallback with a 0.60 similarity cutoff. In request 20 a customer asked for 10,000 **tickets**. Beaver's Choice does not sell tickets — but "*tickets*" scores **0.632** against "*sticky notes*", over the cutoff. The system resolved it, sold 10,000 sticky notes, and billed the customer \$255 for a product they had never mentioned. The reply called the line "Tickets" while the ledger said "Sticky notes".

Auditing the full corpus of 74 item phrasings the agents had used across all runs showed this was not isolated. The 0.60 cutoff was also mapping:

the customer said	they were sold
A4 printing paper	Wrapping paper
A4 white printer paper	Glitter paper
printer paper	Poster paper
A4 size printer paper	220 gsm poster paper
decorative washi tape	Decorative paper
tickets	Sticky notes

A separate case survived even at a raised cutoff: "*A3 paper*" scores **0.875** against "*A4 paper*", and request 3 asks for 5,000 sheets of A3. Swapping one named paper size for another is a substitution, not a spelling correction.

The resolver was rebuilt as the ordered pipeline in section 1.5: a word-level blocklist ahead of any scoring, a curated synonym table matched both exactly and by containment, a cutoff raised to 0.82, and a rule that fuzzy matching may never cross a paper-size boundary. Across the 74-phrase corpus, 70 now resolve and the four rejections — tickets, balloons, cardboard for signage, A3 paper — are all genuinely not carried.

`audit_reconciliation.py` was written to make this class of failure visible rather than merely fixed. It partitions the ledger per request by walking prefix sums against each row's cash balance, then checks that the cash delta equals sales minus restocks for that slice, that each *Confirmed order* total equals both its line items and the ledger, and that no `record_sale` call resolved to a term the catalog does not carry. Run against the pre-fix log it flags exactly one item — `tickets` — and nothing else.

The pattern across all four defects. Each was caused by a component quietly substituting its own judgement for a fact it should have looked up: a price carried in the model's memory, a status token copied into prose, a total recalculated from recollection, a product name guessed by string similarity. Each was fixed by moving the guarantee into code at the point of use. Stated once: *a value that must be*

correct should be computed where it is used, not passed between agents — and where no correct value exists, the system must refuse rather than approximate.

4.9 Remaining weaknesses

1. **The prose and the ledger block can disagree in tone.** When the pricing guard fires, the customer sees an em dash mid-sentence followed by a correct summary. It is never wrong, but it is not elegant — the real fix is 5.2.
2. **The synonym table is curated from observed traffic.** `CATALOG_ALIASES` was built from wording that actually appeared in requests. A customer using an unanticipated synonym will be told the item is not carried — a safe failure, but a lost sale. Section 5.2 addresses this.
3. **Replenishment is purely reactive.** Stock is only ordered once a customer has asked for it, by which point the supplier lead time frequently exceeds the deadline. Inventory value fell over the period despite 28 restock orders, and 12 further restocks were refused on timing.
4. **Two copies of the pricing formula.** `calculate_quote` and `record_sale` each compute the line total independently. They agree today and `audit_prices.py` proves it, but a change to one without the other would silently reintroduce 4.5.
5. **Quantities still pass through the model.** `record_sale` re-derives the price but takes the quantity as given. Section 5.3 proposes closing that gap.
6. **Cost and latency.** Four agents and several LLM calls per request meant roughly 2–3 minutes per request, about 50 minutes for a full run.

5. Suggestions for further improvement

5.1 Replace reactive restocking with a predictive reorder policy

`ensure_stock_available` only buys stock after a customer has asked for something we do not have — by which time the supplier lead time usually exceeds their deadline. Both outright declines and most partial ones trace to this.

The starter data already contains what is needed: the `inventory` table carries `min_stock_level` per item, and `transactions` gives sales velocity. A scheduled reorder pass — run between requests, or as a fifth agent using the one remaining slot in the five-agent budget — could reorder any item whose projected stock at the end of its lead time falls below `min_stock_level`. Because the declines were caused by *timing* rather than by price or solvency, this alone would convert declined revenue into sales without touching the pricing logic.

5.2 Compose the whole reply from a template, and resolve items before the agents run

Two halves of the same idea: take the remaining judgement calls out of the model's hands at both edges of the pipeline.

At the **front**, item resolution should happen once, deterministically, before any agent runs — parsing the request into `(item, quantity, required_by)` tuples with `_resolve_item_name` and surfacing unresolved items immediately. That would also let an unrecognised synonym be logged for review and added to `CATALOG_ALIASES`, turning weakness 4.9.2 into a feedback loop rather than a silent lost sale.

At the **back**, section 4.7 fixed pricing by generating it from the ledger, but the surrounding prose is still authored by the model, which is why a corrected reply can read awkwardly. `handle_customer_request` should assemble the whole message from a structured result — `(item, requested_qty, fulfilled_qty, total, discount_rate, availability_date, decline_reason)` — rendered through a Python template, asserting that every requested line has a verdict. The model would still do the language understanding and the judgement in the middle; it would simply stop being the last thing between the data and the customer.

5.3 Make sales structured and quantity-safe, and share the pricing formula

`record_sale` re-derives the price but accepts the quantity it is handed, one line at a time. Replacing the free-form handoff with a single bulk sales tool taking structured JSON — the whole basket in one call, each entry carrying the originally requested quantity — would let the tool assert that what is being sold matches what was quoted, and would remove the per-line drift where a 5,000-unit request can be recorded as 2,000.

The same change should extract the shared computation into a single `_line_total(item_name, quantity)` used by both `calculate_quote` and `record_sale`, so quote and ledger are identical by construction rather than by audit (weakness 4.9.4). While there, the discount ladder could become margin-aware: `BULK_DISCOUNT_TIERS` is currently a flat function of quantity, applied identically to a \$0.02 napkin and a \$2.50 roll of banner paper, and nothing prevents an unprofitable quote if `SUPPLIER_COST_RATIO` were raised. A floor of, say, 20% gross would make the observed 43.4% margin a controlled outcome rather than a fortunate one.

6. Files submitted

File	Contents
<code>diagram/beavers_choice_workflow.png</code>	Agent workflow diagram — 4 agents, 14 tool bindings, each labelled with its purpose and the starter helper function it wraps
<code>project_starter.py</code>	Complete implementation in a single Python file
<code>test_results.csv</code>	Evaluation output over all 20 sample requests
<code>report/reflection_report.pdf</code>	This report

Supporting evidence, not part of the required submission: `run_full.log` (full agent trace), the four verification scripts (`analyze_results.py`, `audit_prices.py`, `audit_replies.py`, `audit_reconciliation.py`), and `run_prev_bad_resolver.log` / `test_results_prev_bad_resolver.csv` — the run in which request 20 sold sticky notes against a ticket order, kept as the before-picture for section 4.8.